CHARTE DE PROJET

Early Civilization — PPII FISE

Les Incivilises

Lead technique	**Maxence Arditti**
IA & Barbares	**Armel Stéphane Novak Ki**
Unités, Combat & Technologie	**Ababacar Diene**
Villes, Ressources & Documentation	**Wassim Oukazi**

Telecom Nancy | Semestre 6 | 2025–2026

Rendu : 29 mai 2026

1. Objet de la charte

Ce document définit les règles de fonctionnement, les conventions techniques et les engagements de l'équipe pour le projet Early Civilization, réalise dans le cadre du Projet Pluridisciplinaire d'Informatique Intégrative 2 (PPII).

Il constitue le contrat interne de l'équipe et doit être respecté par chacun des membres du groupe. Toute modification doit faire l'objet d'un accord collectif et être tracée dans un compte rendu de réunion.

2. Composition de l'equipe

Prenom Nom	Rôle principal	Périmètre technique
Maxence Arditti	**Lead technique**	Architecture, carte, CLI, SDL, parser
Armel Stéphane Novak Ki	**IA & Barbares**	Makefile, barbares, IA, extension
Ababacar Diene	**Unités, Combat & Technologie**	Unités, combat, arbre technologique
Wassim Oukazi	**Villes, Ressources & Documentation**	Villes, ressources, bâtiments, évènements

3. Organisation du travail

3.1 Réunions

L'équipe se retrouve en réunion hebdomadaire, chaque semaine, pour faire le point sur l'avancement, lever les blocages et planifier la semaine suivante. Chaque réunion donne lieu a un compte rendu rédigé par le secrétaire de séance et déposé sur le dépôt GitLab dans le dossier docs/ dans les 24 heures.

Une réunion extraordinaire peut être convoquée a tout moment par n'importe quel membre en cas de blocage technique ou de retard significatif.

3.2 Gestion des taches

Les taches sont reparties conformément au diagramme de Gantt prévisionnel. Chaque membre est responsable de ses taches et s'engage a signaler tout retard ou difficulté au reste de l'équipe des que possible, sans attendre la réunion suivante.

En cas d'empêchement exceptionnel, le membre concerné prévient le groupe et la tache est redistribuée collectivement.

3.3 Communication

Les échanges quotidiens se font via le serveur Discord de l'équipe. Les fils de discussion sont organises par thématique. Toute décision importante prise hors réunion doit être résumée dans le canal général et tracée dans le prochain compte rendu.

4. Conventions Git

4.1 Branches

L'équipe adopte un workflow base sur les branches de fonctionnalité (feature branching) :

- main : branche stable uniquement. Ne recevoir que des merges valides.
- develop : branche d'integration. Toutes les features y sont mergees avant main.
- feature/<nom> : une branche par fonctionnalite ou tache. Exemple : feature/combat, feature/arbre-tech.
- fix/<nom> : pour les corrections de bugs. Exemple : fix/famine-crash.

Les branches de feature sont créées depuis develop et mergees dans develop via une Merge Request GitLab.

4.2 Messages de commit

Les messages de commit suivent la convention suivante :

```
type: description courte en minuscules
```

Types autorises :

- feat : nouvelle fonctionnalité
- fix : correction de bug
- test : ajout ou modification de tests
- doc : documentation uniquement
- refactor : réorganisation du code sans changement de comportement
- chore : taches annexes (Makefile, .gitignore, etc.)

Exemples valides : feat: implement unit movement, fix: city founding crash on water tile, test: add combat resolution tests

4.3 Merge Requests

Toute Merge Request doit être relue par au moins un autre membre de l'équipe avant d'être acceptée. La relecture porte sur la correction du code, le respect des conventions et l'absence de régression sur les fonctionnalités existantes.

5. Conventions de code

5.1 Nommage

Élement	Convention	Exemple
Variables & fonctions	snake_case	`compute_city_revenue()`
Constantes & macros	UPPER_SNAKE_CASE	`MAX_CITIES, DEFAULT_WIDTH`
Types & structs	PascalCase avec typedef	`typedef struct { ... } City;`
Fichiers	snake_case	`city.c / city.h`

5.2 Structure des fichiers

Chaque module donne lieu a une paire de fichiers : un fichier d'en-tête .h (interface publique, structs, prototypes) et un fichier source .c (implémentation). Les fichiers .h ne contiennent aucune logique d'affichage (CLI ou SDL).

- include/ : tous les fichiers .h
- src/ : tous les fichiers .c
- tests/ : fichiers de tests unitaires
- docs/ : comptes rendus, spécifications, planning
- assets/ : ressources graphiques éventuelles

5.3 Commentaires

Chaque fonction publique (déclarée dans un .h) est documentée par un commentaire en français au-dessus de sa déclaration, précisent son rôle, ses paramètres et sa valeur de retour. Le code interne peut être commente en anglais ou en français, de manière cohérente au sein d'un même fichier.

Les blocs TODO doivent être préfixés par // TODO: et mentionner le nom du responsable.

5.4 Qualité

- Variables globales interdites sauf constantes de configuration.
- Toute allocation mémoire dynamique (malloc/calloc) doit avoir une libération (free) correspondante.
- Les entrées invalides en ligne de commande déclenchent l'affichage de --help et une sortie propre.
- Le code doit compiler sans avertissement avec les flags : -Wall -Wextra -std=c11.

6. Tests

Des tests unitaires couvrant le cœur du moteur de jeu (carte, villes, combat, technologie, barbares) sont rédiges et maintenus tout au long du projet. Ils sont exécutables via make test et ne doivent pas produire d'erreurs ou de fuites mémoire détectes par Valgrind.

Chaque fonctionnalité implémentée doit être accompagnée d'au moins un test couvrant un cas nominal et un cas limite.

7. Livrables et jalons

Date	Jalon	Contenu attendu
~21 avr. 2026	**MVP CLI**	Partie jouable de bout en bout en mode terminal
~5 mai 2026	**Stabilisation**	Tests unitaires, IA barbares consolidée, extension(s) implémentées
~19 mai 2026	**SDL + finalisation**	Interface graphique SDL fonctionnelle, débogage valgrind
29 mai 2026 22h	**Rendu final**	Depot GitLab complet : code, Makefile, tests, docs, README

8. Règles de vie de l'équipe

8.1 Engagements individuels

- Respecter les échéances fixées en réunion et informer l'équipe sans délai en cas de difficulté.
- Ne jamais merger sur main ou develop sans Merge Request validee.
- Ne pas modifier le code d'un autre membre sans le prévenir au préalable.
- Assurer la lisibilité et la maintenabilité de son propre code avant tout commit.
- Déclarer explicitement toute utilisation d'IA générative dans les commentaires et le rapport, conformément aux règles du module.

8.2 Resolution des désaccords

En cas de désaccord technique ou organisationnel, la discussion est ouverte lors de la prochaine réunion (ou en urgence sur Discord). Si aucun consensus n'est trouve, la décision est soumise a un vote a la majorité simple. Le résultat est consigne dans le compte rendu.

8.3 Non-respect des règles

En cas de non-respect avéré d'une règle de la présente charte, l'équipe organise une discussion collective pour comprendre la situation et trouver une solution. L'évènement est mentionné dans le compte rendu de réunion. Aucune sanction formelle n'est appliquée au-delà de cette discussion, dans l'esprit d'une résolution constructive et bienveillante.

9. Adoption de la charte

Les membres soussignés s'engagent a respecter les règles déifies dans ce document pour la durée du projet.

Maxence Arditti Lead technique	**Armel Stéphane Novak Ki** IA & Barbares	**Ababacar Diene** Unités, Combat & Technologie	**Wassim Oukazi** Villes, Ressources & Documentation
Signature :	Signature :	Signature :	Signature :

Fait a Nancy, le _____ / _____ / 2026